This code deleted all but four of our `<g>` elements, because there are only four values in our array. In most of the explanations of D3's `.enter()` and `.exit()` behavior, you won't see this kind of binding of an entirely different array to a selection. Instead, you'll see a rebinding of the initial data array after it's been filtered to represent a change via user interaction or other behavior. You'll see an example like this next, and throughout the book. But it's important to understand the difference between your data, your selection, and your DOM elements. The data that's bound to our DOM elements has been overwritten, so our data-rich objects from tweets.csv have now been replaced with boring numbers. But the only change to the visual representation is that the number has been reduced to reflect the size of the array we've bound. D3 doesn't follow the convention that when the data changes, the corresponding display is updated; you need to build that functionality yourself. Because it doesn't follow that convention, it gives you greater flexibility that we'll explore in later chapters.

UPDATING

You can see how the visual attributes of an element can change to reflect changes in data by updating the `<text>` elements in each `g` to reflect the newly bound data:

```
d3.selectAll("g").select("text").text(function(d) {return d});
```

Figure 2.26 shows our long labels replaced by the numbers we bound to the data.

In this example we had to `.selectAll()` the parent elements and then subselect the child elements to re-initialize the data-binding for the child elements. Whenever you bind new data to a selection that utilizes child elements, you'll need to follow this pattern. You can see that, because we didn't update the `<circle>` elements, they still have the old data bound to each element:

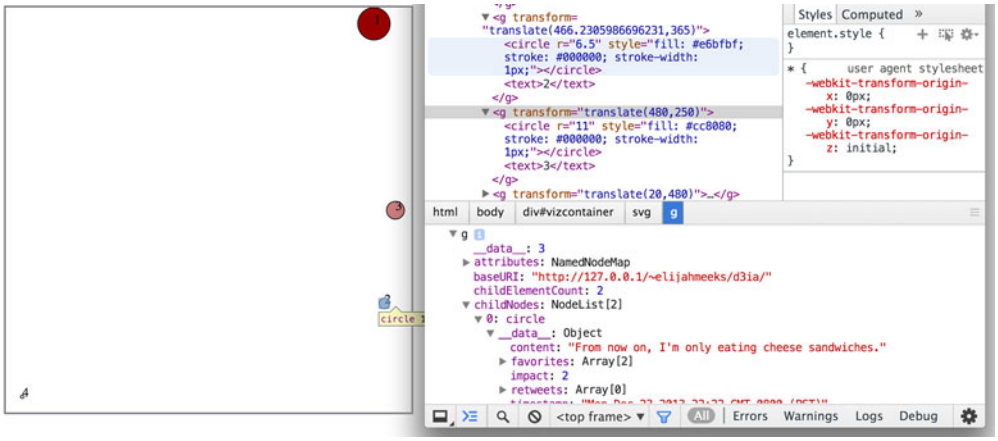


Figure 2.26 Only four `<g>` elements remain, corresponding to the four data values in the new array, with their `<text>` labels reset to match the new values in the array. But when you inspect the `<g>` element, you see that its `__data__` property, where D3 stores the bound data, is different from that of its `<circle>` child element, which still has the JSON object we bound when we first created the visualization.